Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 6

По дисциплине “Защита информации”

Выполнил: студент группы ИВТ-42-23

Иванов И.Л.

Проверил: кандидат технических наук

Ковалев С.В.

Чебоксары, 2026

1. Цель работы:

Целью работы является знакомство с особенностями работы алгоритма DES в режиме CFB.

2. Текст программы.

Обобщенная схема шифрования DES в режиме CFB приведена на рисунке 1, схема дешифрования — на рисунке 2.

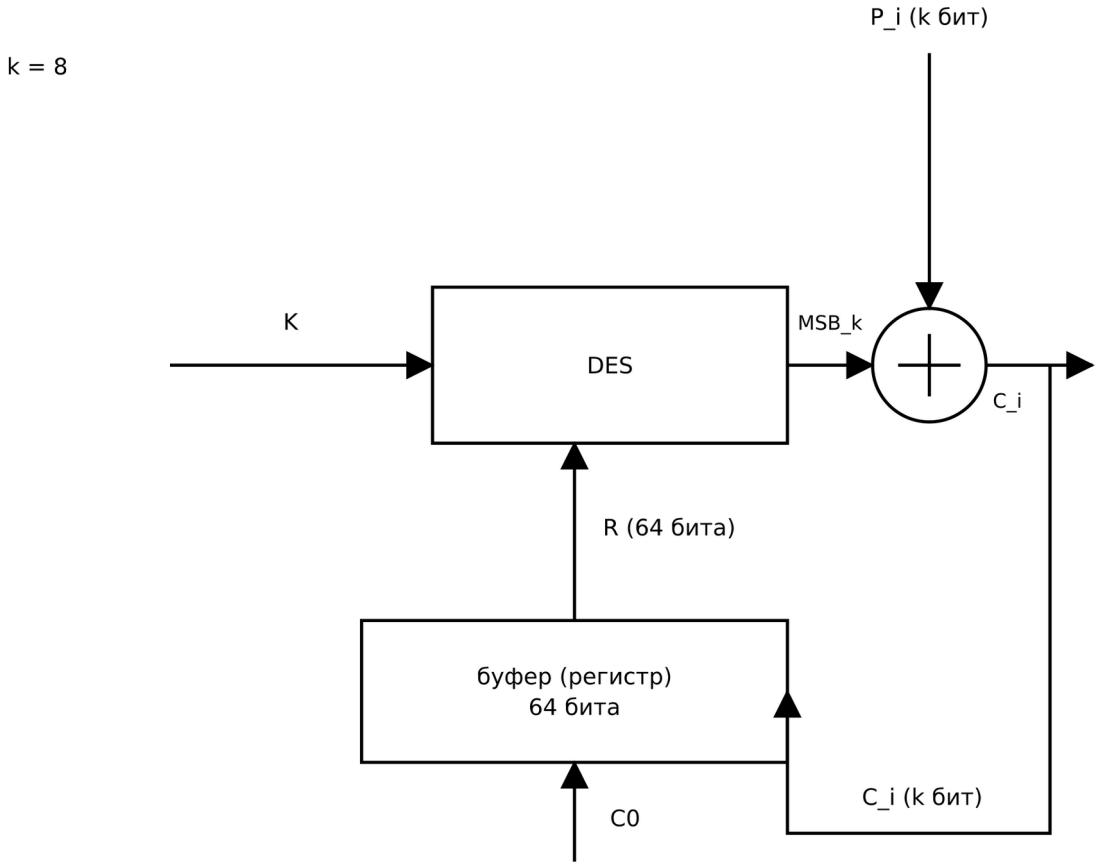


Рисунок 1 – Схема шифрования DES в режиме CFB

Блок-схемы алгоритмов выполнены в соответствии с ГОСТ 19.701-90.

Программа реализована на языке Go и не использует библиотечных реализаций блочного шифрования: все преобразования алгоритма DES выполнены непосредственно в коде. Базовая реализация DES — таблицы IP, IP^{-1}, E, S1–S8, P, PC-1, PC-2, LS, функции GenerateKeys, Feistel,

EncryptBlock и DecryptBlock — перенесена из лабораторных работ №4 и №5. Программа откомпилирована и проверена; результаты ее выполнения приведены в пп. 3–5.

В состав программы входят следующие функциональные элементы.

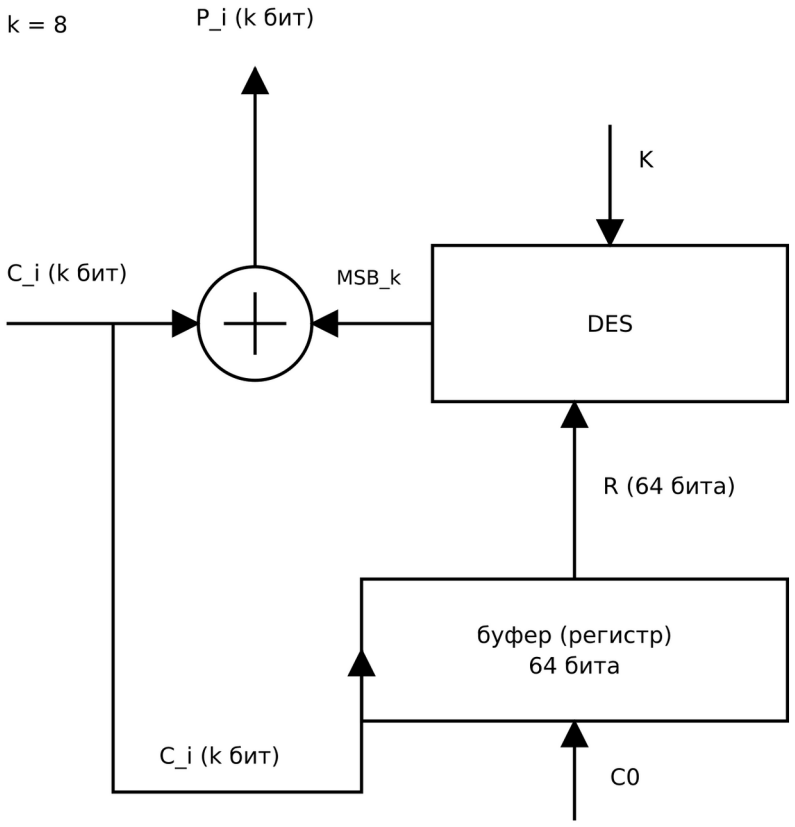


Рисунок 2 – Схема дешифрования DES в режиме CFB

Режим CFB. Функция EncryptCFB обрабатывает открытый текст побайтово, то есть при k = 8. Для каждого байта вычисляется значение Oi = DES_K(Ri-1), затем Ci = Pi ⊕ MSB8(Oi) и выполняется сдвиг регистра: Ri = (Ri-1 << 8) | Ci. Функция DecryptCFB выполняет обратное преобразование: Pi = Ci ⊕ MSB8(DES_K(Ri-1)); в регистр обратной связи записывается байт шифртекста Ci. Существенно, что в режиме CFB при

дешифровании используется та же функция EncryptBlock, что и при шифровании, а не DecryptBlock.

Генератор начального вектора. 64-битовый начальный вектор C0 формируется датчиком ПСЧ, разработанным в лабораторной работе №3. Использован линейный конгруэнтный генератор T(i+1) = (5·T(i) + 51) mod 256 с начальным состоянием T(0) = 13. Из восьми последовательных значений генератора получен вектор C0 = 0x747786D1489B3A55.

Дополнение по стандарту PKCS#7. В режиме CFB дополнение не используется: режим обрабатывает данные как поток байтов, поэтому длина шифрограммы равна длине открытого текста. Функции PKCS7Pad и PKCS7Unpad сохранены в программе как вспомогательные и в основном тракте CFB не применяются.

Вспомогательные режимы. Функции EncryptECB и EncryptCBC сохранены для сравнительного эксперимента в п. 5, где одинаковые блоки открытого текста шифруются в трех режимах. В основном тракте программы они не используются.

Управляющая функция main последовательно выполняет три эксперимента: базовое шифрование и дешифрование текста в режиме CFB; искажение одного бита шифрограммы и дешифрование искаженных данных; шифрование текста с двумя одинаковыми 64-битными блоками в режимах ECB, CBC и CFB.

Блок-схема алгоритма шифрования в режиме CFB приведена на рисунке 3.

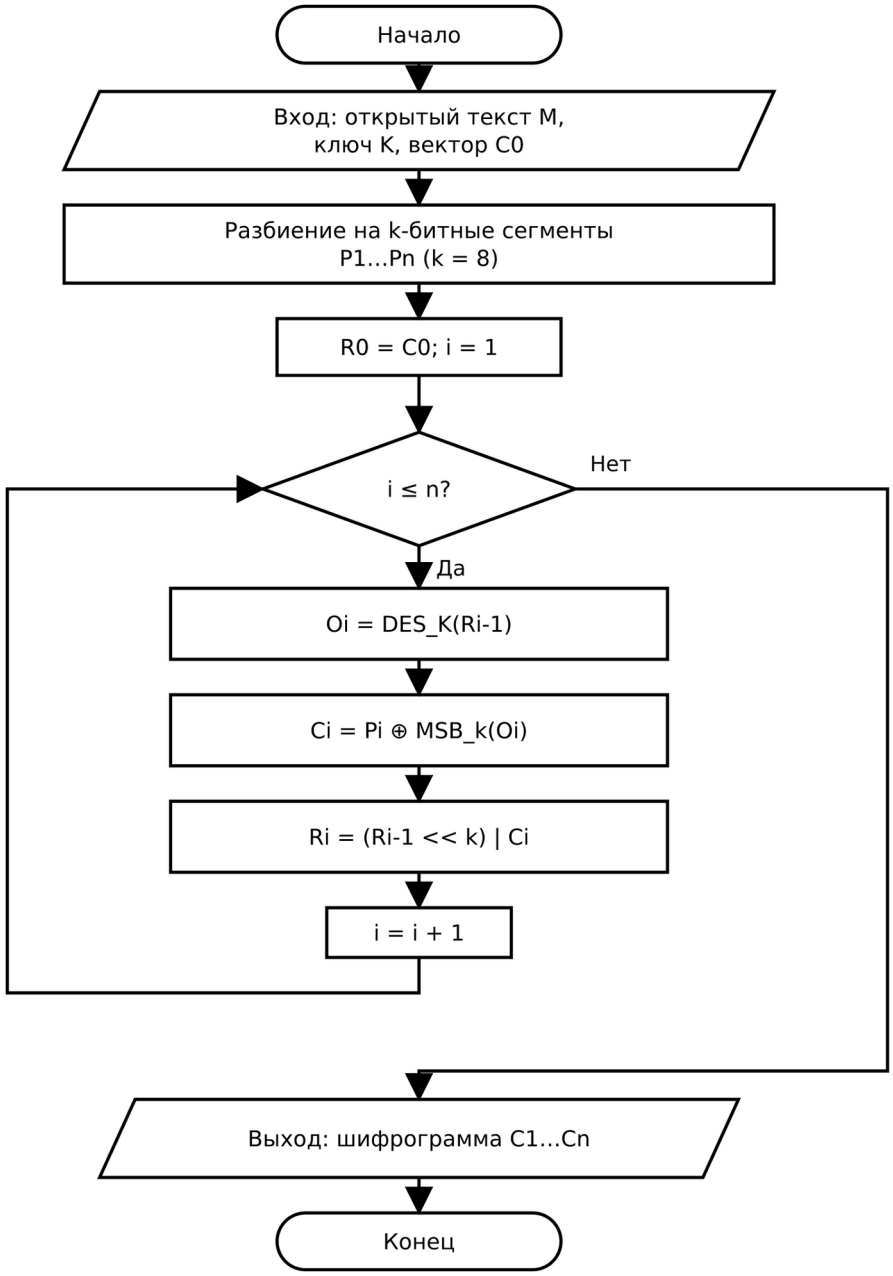


Рисунок 3 – Блок-схема алгоритма шифрования в режиме CFB

Ниже приведен полный листинг программы (файл lab6/main.go).

https://github.com/kukirmash/labs/tree/main/7sem/ZI/lab6

```
package main

import (
    "bytes"
    "encoding/binary"
    "errors"
    "fmt"
    "math/bits"
)
```

```
//
-----------------------------------------------------------------------------------------------
---
// IP — таблица начальной перестановки (Initial Permutation) с нумерацией битов от 1 до 64
var IP = [64]int{
    58, 50, 42, 34, 26, 18, 10, 2,
    60, 52, 44, 36, 28, 20, 12, 4,
    62, 54, 46, 38, 30, 22, 14, 6,
    64, 56, 48, 40, 32, 24, 16, 8,
    57, 49, 41, 33, 25, 17, 9, 1,
    59, 51, 43, 35, 27, 19, 11, 3,
    61, 53, 45, 37, 29, 21, 13, 5,
    63, 55, 47, 39, 31, 23, 15, 7,
}


// IPInv — таблица 2 обратной конечной перестановки (IP-1)
var IPInv = [64]int{
    40, 8, 48, 16, 56, 24, 64, 32,
    39, 7, 47, 15, 55, 23, 63, 31,
    38, 6, 46, 14, 54, 22, 62, 30,
    37, 5, 45, 13, 53, 21, 61, 29,
    36, 4, 44, 12, 52, 20, 60, 28,
    35, 3, 43, 11, 51, 19, 59, 27,
    34, 2, 42, 10, 50, 18, 58, 26,
    33, 1, 41, 9, 49, 17, 57, 25,
}


//
-----------------------------------------------------------------------------------------------
---
// E — таблица 3 расширения (32 бита -> 48 бит)
var E = [48]int{
    32, 1, 2, 3, 4, 5,
    4, 5, 6, 7, 8, 9,
    8, 9, 10, 11, 12, 13,
    12, 13, 14, 15, 16, 17,
    16, 17, 18, 19, 20, 21,
    20, 21, 22, 23, 24, 25,
    24, 25, 26, 27, 28, 29,
    28, 29, 30, 31, 32, 1,
}


//
-----------------------------------------------------------------------------------------------
---
// SBox — таблицы 4 нелинейных подстановок (8 блоков по 4 строки и 16 столбцов)
var SBox = [8][4][16]byte{
    { // S1
        {14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7},
        {0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8},
        {4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0},
        {15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13},
    },
    { // S2
        {15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10},
        {3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5},
        {0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15},
        {13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9},
    },
    { // S3
        {10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8},
        {13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1},
        {13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7},
        {1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12},
    },
    { // S4
        {7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15},
        {13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9},
        {10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4},
        {3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14},
    },
    { // S5
        {2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9},
        {14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6},
```

```
        {4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14},
        {11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3},
    },
    { // S6
        {12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11},
        {10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8},
        {9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6},
        {4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13},
    },
    { // S7
        {4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1},
        {13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6},
        {1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2},
        {6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12},
    },
    { // S8
        {13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7},
        {1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2},
        {7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 13, 5, 8},
        {2, 1, 14, 7, 4, 10, 8, 13, 5, 12, 9, 0, 3, 5, 5, 11},
    },
}

//
---------------------------------------------------------------------------------------------
---
// P — таблица 5 перестановки внутри функции Фейстеля
var P = [32]int{
    16, 7, 20, 21, 29, 12, 28, 17, 1, 15, 23, 26, 5, 18, 31, 10,
    2, 8, 24, 14, 32, 27, 3, 9, 19, 13, 30, 6, 22, 11, 4, 25,
}


//
---------------------------------------------------------------------------------------------
---
// PC1 — таблица 6 перестановки-выбора PC-1 (56 элементов, нумерация битов от 1 до 64)
var PC1 = [56]int{
    57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18,
    10, 2, 59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36,
    63, 55, 47, 39, 31, 23, 15, 7, 62, 54, 46, 38, 30, 22,
    14, 6, 61, 53, 45, 37, 29, 21, 13, 5, 28, 20, 12, 4,
}


//
---------------------------------------------------------------------------------------------
---
// LS — таблица 7 циклических сдвигов для 16 раундов
var LS = [16]int{
    1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1,
}


//
---------------------------------------------------------------------------------------------
---
// PC2 — таблица 8 перестановки-выбора PC-2 (48 элементов)
var PC2 = [48]int{
    14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10,
    23, 19, 12, 4, 26, 8, 16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48,
    44, 49, 39, 56, 34, 53, 46, 42, 50, 36, 29, 32,
}


//
---------------------------------------------------------------------------------------------
---
// PKCS7Pad дополняет срез байт до кратности blockSize (для DES это 8 байт / 64 бита)
func PKCS7Pad(data []byte, blockSize int) []byte {
    //
    padding := blockSize - (len(data) % blockSize)


    padBytes := bytes.Repeat([]byte{byte(padding)}, padding)
    return append(data, padBytes...)
```

```
}

//
------------------------------------------------------------------------------------------
---
// PKCS7Unpad удаляет дополнение PKCS#7 после дешифрования
func PKCS7Unpad(data []byte) ([]byte, error) {
    length := len(data)
    if length == 0 || length%8 != 0 {
        return nil, errors.New("некорректная длина данных для снятия дополнения")
    }


    padLen := int(data[length-1])
    if padLen == 0 || padLen > 8 {
        return nil, errors.New("некорректное значение байта дополнения")
    }


    for i := length - padLen; i < length; i++ {
        if data[i] != byte(padLen) {
            return nil, errors.New("нарушена целостность байтов дополнения")
        }
    }


    return data[:length-padLen], nil
}


//
------------------------------------------------------------------------------------------
---
// 64 -> 64
func PermuteIP(val uint64) uint64 {
    var res uint64


    for i := 0; i < 64; i++ {
        pos := IP[i]


        bit := (val >> (64 - pos)) & 1


        res |= bit << (63 - i)
    }
    return res
}


//
------------------------------------------------------------------------------------------
---
func PermuteIPInv(val uint64) uint64 {
    var res uint64


    for i := 0; i < 64; i++ {
        pos := IPInv[i]
        bit := (val >> (64 - pos)) & 1
        res |= bit << (63 - i)
    }
    return res
}


//
------------------------------------------------------------------------------------------
---
// 64 -> 56
func PermutePC1(key uint64) uint64 {
    var key56 uint64


    for i := 0; i < 56; i++ {
        // Номер бита в 64-битном ключе (от 1 до 64)
        pos := PC1[i]
```

```
        // Извлекаем бит из ключа (1-й бит — самый старший, сдвиг на 64 - pos)
        bit := (key >> (64 - pos)) & 1

        // Укладываем бит в 56-битное число от старшего к младшему (55 - i)
        key56 |= bit << (55 - i)
    }

    return key56
}

//
------------------------------------------------------------------------------------------------
---
// 56 -> 48
func PermutePC2(key uint64) uint64 {

    var key48 uint64
    for i := 0; i < 48; i++ {
        // Номер бита в 56-битном векторе (от 1 до 56)
        pos := PC2[i]

        // Извлекаем нужный бит. 1-й бит вектора cd — старший (находится на позиции 55)
        bit := (key >> (56 - pos)) & 1

        // Укладываем бит в 48-битный раундовый ключ от старшего к младшему (сдвиг 47 - i)
        key48 |= bit << (47 - i)
    }

    return key48
}

//
------------------------------------------------------------------------------------------------
---
// 32 -> 48
func PermuteE(val uint32) uint64 {
    var res uint64

    for i := 0; i < 48; i++ {
        pos := E[i]

        bit := (val >> (32 - pos)) & 1

        res |= uint64(bit) << (47 - i)
    }
    return res
}

//
------------------------------------------------------------------------------------------------
---
// 32 -> 32
func PermuteP(val uint32) uint32 {
    var res uint32

    for i := 0; i < 32; i++ {
        pos := P[i]

        bit := (val >> (32 - pos)) & 1

        res |= bit << (31 - i)
    }
    return res
```

```
}

//
-----------------------------------------------------------------------------------------
---
func GenerateKeys(srcKey uint64) [16]uint64 {
    var key56 uint64 = PermutePC1(srcKey)


    // L0 (в коде - левая часть) — это старшие 28 бит. Сдвигаем вправо на 28 позиций.
    var L uint32 = uint32(key56 >> 28)


    // R0 (в коде - правая часть) — это младшие 28 бит. Оставляем только их с помощью маски.
    // 0x0FFFFFFF в двоичном виде — это ровно 28 единиц.
    var R uint32 = uint32(key56 & 0x0FFFFFFF)


    var keys [16]uint64


    for i := 0; i < 16; i++ {


        shift := LS[i]


        // Выполняем циклический сдвиг влево на shift позиций для каждой 28-битной половины.
        // Конструкция (val >> (28 - shift)) переносит старшие выпадающие биты в младшие
разряды.
        L = ((L << shift) | (L >> (28 - shift))) & 0x0FFFFFFF
        R = ((R << shift) | (R >> (28 - shift))) & 0x0FFFFFFF


        // Склеиваем сдвинутые половины обратно в единый 56-битный вектор.
        // Левая половина сдвигается на 28 позиций влево, правая добавляется через "ИЛИ".
        cd := (uint64(L) << 28) | uint64(R)


        // Применяем перестановку-выбор PC-2 для получения 48-битного ключа
        keys[i] = PermutePC2(cd)
    }


    return keys
}


//
-----------------------------------------------------------------------------------------
---
// Feistel выполняет функцию f(R, K) для одного раунда
func Feistel(r uint32, key uint64) uint32 {


    // Расширение
    var e uint64 = PermuteE(r)


    // Гаммирование
    B := e ^ key


    // Преобразование в S-блоках (48 бит -> 32 бита)
    var s uint32
    for i := 0; i < 8; i++ {


        // 6-битный вектор
        vector6 := byte((B >> (42 - i*6)) & 0x3F)


        // 1,6
        k := ((vector6 >> 4) & 0x2) | (vector6 & 0x1)
        // 2,3,4,5
        l := (vector6 >> 1) & 0xF


        // Получаем 4-битное значение из S-блока
```

```
        val4 := uint32(SBox[i][k][l])

        // Укладываем 4 бита в итоговый 32-битный вектор
        s |= val4 << (28 - i*4)
    }

    // Перестановка P
    result := PermuteP(s)

    return result
}

//
-----------------------------------------------------------------------------------------------
---
// EncryptBlock шифрует один 64-битный блок
func EncryptBlock(block uint64, keys [16]uint64) uint64 {
    // Начальная перестановка
    block = PermuteIP(block)

    // Разделение на левую и правую 32-битные половины
    L := uint32(block >> 32)
    R := uint32(block & 0xFFFFFFFF)

    // 16 раундов сети Фейстеля
    for i := 0; i < 16; i++ {
        nextL := R
        nextR := L ^ Feistel(R, keys[i])

        L = nextL
        R = nextR
    }

    // Склейка половин в обратном порядке (R16, L16)
    reversedBlock := (uint64(R) << 32) | uint64(L)

    // Конечная обратная перестановка
    return PermuteIPInv(reversedBlock)
}

//
-----------------------------------------------------------------------------------------------
---
// DecryptBlock расшифровывает один 64-битный блок
func DecryptBlock(block uint64, keys [16]uint64) uint64 {
    block = PermuteIP(block)

    L := uint32(block >> 32)
    R := uint32(block & 0xFFFFFFFF)

    // Те же 16 раундов, но ключи идут в обратном порядке (от 15 к 0)
    for i := 15; i >= 0; i-- {
        nextL := R
        nextR := L ^ Feistel(R, keys[i])

        L = nextL
        R = nextR
    }

    reversedBlock := (uint64(R) << 32) | uint64(L)

    return PermuteIPInv(reversedBlock)
}
```

```
//
-----------------------------------------------------------------------------------------------
---
// EncryptECB шифрует срез байт (кратный 8) в режиме ECB
func EncryptECB(data []byte, keys [16]uint64) []byte {
    encrypted := make([]byte, len(data))
    for i := 0; i < len(data); i += 8 {
        block := binary.BigEndian.Uint64(data[i : i+8])
        encryptedBlock := EncryptBlock(block, keys)
        binary.BigEndian.PutUint64(encrypted[i:i+8], encryptedBlock)
    }
    return encrypted
}


//
-----------------------------------------------------------------------------------------------
---
// DecryptECB расшифровывает срез байт (кратный 8) в режиме ECB
func DecryptECB(cipherData []byte, keys [16]uint64) []byte {
    decrypted := make([]byte, len(cipherData))
    for i := 0; i < len(cipherData); i += 8 {
        block := binary.BigEndian.Uint64(cipherData[i : i+8])
        decryptedBlock := DecryptBlock(block, keys)
        binary.BigEndian.PutUint64(decrypted[i:i+8], decryptedBlock)
    }
    return decrypted
}


//
-----------------------------------------------------------------------------------------------
---
// EncryptCBC шифрует срез байт (кратный 8) в режиме CBC
func EncryptCBC(data []byte, keys [16]uint64, iv uint64) []byte {
    if len(data)%8 != 0 {
        panic("длина данных должна быть кратна 8 байтам")
    }


    encrypted := make([]byte, len(data))
    prev := iv


    for i := 0; i < len(data); i += 8 {
        block := binary.BigEndian.Uint64(data[i : i+8])
        encryptedBlock := EncryptBlock(block^prev, keys)
        binary.BigEndian.PutUint64(encrypted[i:i+8], encryptedBlock)


        // Каждый следующий блок сцепляется с предыдущим блоком шифртекста
        prev = encryptedBlock
    }


    return encrypted
}


//
-----------------------------------------------------------------------------------------------
---
// DecryptCBC расшифровывает срез байт (кратный 8) в режиме CBC
func DecryptCBC(cipherData []byte, keys [16]uint64, iv uint64) []byte {
    if len(cipherData)%8 != 0 {
        panic("длина шифртекста должна быть кратна 8 байтам")
    }


    decrypted := make([]byte, len(cipherData))
    prev := iv


    for i := 0; i < len(cipherData); i += 8 {
        block := binary.BigEndian.Uint64(cipherData[i : i+8])
        decryptedBlock := DecryptBlock(block, keys) ^ prev
        binary.BigEndian.PutUint64(decrypted[i:i+8], decryptedBlock)
```

```
        // Для следующего блока предыдущим является текущий блок шифртекста
        prev = block
    }

    return decrypted
}

//
-----------------------------------------------------------------------------------------
---
// GenerateC формирует 64-битный начальный вектор C0 из восьми значений ПСЧ
// T(i+1) = (5 * T(i) + 51) mod 256
func GenerateC(C0 uint8) uint64 {
    var c uint64
    var prevVal uint8 = C0

    for i := 0; i < 8; i++ {
        valByte := byte((5*int(prevVal) + 51) % 256)
        prevVal = valByte

        // Складываем в вектор
        c = (c << 8) | uint64(valByte)
    }

    return c
}

//
-----------------------------------------------------------------------------------------
---
// DiffBits возвращает количество различающихся бит в двух 64-битных блоках
func DiffBits(a uint64, b uint64) int {
    return bits.OnesCount64(a ^ b)
}

//
-----------------------------------------------------------------------------------------
---
// EncryptCFB шифрует данные в режиме CFB с k = 8 бит (побайтовая обработка).
// R0 = C0; Oi = DES_K(Ri-1); Ci = Mi XOR MSB_8(Oi); Ri = (Ri-1 << 8) | Ci.
func EncryptCFB(data []byte, keys [16]uint64, iv uint64) []byte {
    encrypted := make([]byte, len(data))
    reg := iv

    for i, p := range data {
        stream := EncryptBlock(reg, keys)
        c := p ^ byte(stream>>56) // левые 8 бит результата DES

        encrypted[i] = c
        reg = (reg << 8) | uint64(c)
    }

    return encrypted
}

//
-----------------------------------------------------------------------------------------
---
// DecryptCFB расшифровывает данные в режиме CFB с k = 8 бит.
// В CFB при дешифровании используется та же функция EncryptBlock, а в регистр
// обратной связи записывается байт шифртекста.
func DecryptCFB(cipherData []byte, keys [16]uint64, iv uint64) []byte {
    decrypted := make([]byte, len(cipherData))
    reg := iv

    for i, c := range cipherData {
        stream := EncryptBlock(reg, keys)
```

```
        decrypted[i] = c ^ byte(stream>>56)

        reg = (reg << 8) | uint64(c)
    }

    return decrypted
}

//
----------------------------------------------------------------------------------------------
---
// DiffBits8 возвращает количество различающихся бит в двух байтах
func DiffBits8(a byte, b byte) int {
    return bits.OnesCount8(a ^ b)
}

//
----------------------------------------------------------------------------------------------
---
func main() {
    str := "Каморка его приходилась под самою кровлей высокого пятиэтажного дома и походила
более на шкаф, чем на квартиру."
    data := []byte(str)

    const srcKey uint64 = 0x13_34_57_79_9B_BC_DF_F1
    keys := GenerateKeys(srcKey)

    // Начальный вектор C0 формируется генератором ПСЧ из ЛР №3.
    const T0 uint8 = 13
    c0 := GenerateC(T0)

    // ========================================================================
    fmt.Println("=== 1. ШИФРОВАНИЕ И ДЕШИФРОВАНИЕ (CFB, k = 8) ===")
    fmt.Println("Исходная строка:", str)
    fmt.Printf("Ключ (HEX): 0x%016X\n", srcKey)
    fmt.Printf("Начальный вектор C0 (HEX): 0x%016X\n", c0)
    fmt.Printf("Длина открытого текста: %d байт; длина шифрограммы: %d байт (без дополнения)
\n",
        len(data), len(data))

    encryptedData := EncryptCFB(data, keys, c0)
    fmt.Printf("Шифртекст (HEX): %X\n", encryptedData)

    decryptedData := DecryptCFB(encryptedData, keys, c0)
    fmt.Println("Расшифрованная строка:", string(decryptedData))
    fmt.Printf("Совпадение с исходным текстом: %v\n", string(decryptedData) == str)
    fmt.Println()

    // ========================================================================
    fmt.Println("=== 2. РАЗМНОЖЕНИЕ ОШИБКИ (искажение 1 бита) ===")

    corruptedCipher := make([]byte, len(encryptedData))
    copy(corruptedCipher, encryptedData)
    corruptedCipher[0] ^= 0x01

    fmt.Printf("Первый байт шифрограммы: %02X -> %02X\n", encryptedData[0],
corruptedCipher[0])
    fmt.Printf("Искаженный шифртекст (HEX): %X\n", corruptedCipher)

    corruptedDecrypted := DecryptCFB(corruptedCipher, keys, c0)
    fmt.Printf("Искаженный открытый текст (HEX): %X\n", corruptedDecrypted)
    fmt.Printf("Искаженный открытый текст: %q\n", string(corruptedDecrypted))

    fmt.Println("Различия по байтам (первые 12):")
    n := 12
    if len(data) < n {
```

```
        n = len(data)
    }
    for i := 0; i < n; i++ {
        fmt.Printf("  байт %2d: %02X -> %02X, %d бит\n",
            i, data[i], corruptedDecrypted[i], DiffBits8(data[i], corruptedDecrypted[i]))
    }


    fmt.Println("Различия по 64-битным блокам:")
    for i := 0; i+8 <= len(data); i += 8 {
        originalBlock := binary.BigEndian.Uint64(data[i : i+8])
        corruptedBlock := binary.BigEndian.Uint64(corruptedDecrypted[i : i+8])
        fmt.Printf("  блок %2d: %016X -> %016X, %d бит\n",
            i/8+1, originalBlock, corruptedBlock, DiffBits(originalBlock, corruptedBlock))
    }
    fmt.Println()


    // ============================================================================
    fmt.Println("=== 3. ОДИНАКОВЫЕ БЛОКИ ОТКРЫТОГО ТЕКСТА ===")


    sameStr := "BLOCK123BLOCK123"
    sameData := []byte(sameStr)


    cfbCipher := EncryptCFB(sameData, keys, c0)
    ecbCipher := EncryptECB(sameData, keys)
    cbcCipher := EncryptCBC(sameData, keys, c0)


    cfbC1 := binary.BigEndian.Uint64(cfbCipher[:8])
    cfbC2 := binary.BigEndian.Uint64(cfbCipher[8:16])
    ecbC1 := binary.BigEndian.Uint64(ecbCipher[:8])
    ecbC2 := binary.BigEndian.Uint64(ecbCipher[8:16])
    cbcC1 := binary.BigEndian.Uint64(cbcCipher[:8])
    cbcC2 := binary.BigEndian.Uint64(cbcCipher[8:16])


    fmt.Println("Исходная строка:", sameStr)
    fmt.Printf("M1 = M2 = %016X\n", binary.BigEndian.Uint64(sameData[:8]))
    fmt.Printf("CFB: C1 = %016X, C2 = %016X, C1 == C2: %v\n", cfbC1, cfbC2, cfbC1 == cfbC2)
    fmt.Printf("ECB: C1 = %016X, C2 = %016X, C1 == C2: %v\n", ecbC1, ecbC2, ecbC1 == ecbC2)
    fmt.Printf("CBC: C1 = %016X, C2 = %016X, C1 == C2: %v\n", cbcC1, cbcC2, cbcC1 == cbcC2)
    fmt.Printf("CFB шифртекст (HEX): %X\n", cfbCipher)
    fmt.Printf("ECB шифртекст (HEX): %X\n", ecbCipher)
    fmt.Printf("CBC шифртекст (HEX): %X\n", cbcCipher)
    fmt.Println()

}
```

3. Пример открытого текста и соответствующей ему шифрограммы.

Базовый тест выполнен на 64-битном ключе K = 0x133457799BBCDFF1. Начальный вектор, сформированный генератором ПСЧ из лабораторной работы №3, равен C0 = 0x747786D1489B3A55.

Открытый текст: «Каморка его приходилась под самою кровлей высокого пятиэтажного дома и походила более на шкаф, чем на квартиру.»

Открытый текст содержит 204 байта. Режим CFB обрабатывает данные побайтово (k = 8), поэтому дополнение не используется; длина шифрограммы также составляет 204 байта.

Полученная шифрограмма (HEX):

```
8BAD794AE1CA15A6D4D5A1F44836806A904489FD90FFEF508C68895A561443E5FFEA86C52FD72506633
3C2A7D54C689F6C59243B86BE35F640E6F4F71974001E344580BB45302C641CD8742E086BD554A11ECF
314D4EA583AA060E4746518A8B3AF131E1890EE3EF00D12ABC1F4548EC68584ACF12A9FCAA437ADC252
8742CDFC04A97877B894D60C22FED0BC59E38DCA256741BD5D0B3CF27A18A777BD79AB9EE733297E40C
A60D92ACFF728B1D39A546B54B47F4F7831DA06B247AB32D832C9A7B0F6D7032ABF74890F52C
```

Результат дешифрования шифрограммы совпал с исходным открытым текстом, что подтверждает корректность реализованных алгоритмов шифрования и дешифрования в режиме CFB.

4. Шифрограмма с искаженным битом и соответствующий искаженный открытый текст.

Для демонстрации свойства размножения ошибок в режиме CFB в шифрограмме, полученной в п. 3, был инвертирован ровно один бит — младший бит первого байта. Значение первого байта изменилось с 8B (10001011 в двоичной записи) на 8A (10001010 в двоичной записи).

Искаженная шифрограмма (HEX):

```
8AAD794AE1CA15A6D4D5A1F44836806A904489FD90FFEF508C68895A561443E5FFEA86C52FD72506633
3C2A7D54C689F6C59243B86BE35F640E6F4F71974001E344580BB45302C641CD8742E086BD554A11ECF
314D4EA583AA060E4746518A8B3AF131E1890EE3EF00D12ABC1F4548EC68584ACF12A9FCAA437ADC252
8742CDFC04A97877B894D60C22FED0BC59E38DCA256741BD5D0B3CF27A18A777BD79AB9EE733297E40C
A60D92ACFF728B1D39A546B54B47F4F7831DA06B247AB32D832C9A7B0F6D7032ABF74890F52C
```

После дешифрования искаженной шифрограммы было выполнено сравнение байтов открытого текста. Различия в первых 12 байтах приведены ниже.

Позиция	Оригинал	С ошибкой	Различий, бит
0	D0	D1	1
1	9A	4A	3
2	D0	3A	5
3	B0	6A	5
4	D0	BC	4
5	BC	BF	2
6	D0	AD	6
7	BE	92	3
8	D1	F1	1
9	80	80	0
10	D0	D0	0
11	BA	BA	0

Различия по 64-битным блокам:

Блок	Оригинал	С ошибкой	Различий, бит
1	D09AD0B0D0BCD0BE	D14A3A6ABCBFAD92	29
2	D180D0BAD0B020D0	F180D0BAD0B020D0	1
3	B5D0B3D0BE20D0BF	B5D0B3D0BE20D0BF	0

Блоки 4–25 расшифрованы без искажений (0 различающихся бит).

Искаженный открытый текст (HEX):

```
D14A3A6ABCBFAD92F180D0BAD0B020D0B5D0B3D0BE20D0BFD180D0B8D185D0BED0B4D0B8D0BBD0B0D18
1D18C20D0BFD0BED0B420D181D0B0D0BCD0BED18E20D0BAD180D0BED0B2D0BBD0B5D0B920D0B2D18BD1
81D0BED0BAD0BED0B3D0BE20D0BFD18FD182D0B8D18DD182D0B0D0B6D0BDD0BED0B3D0BE20D0B4D0BED
0BCD0B020D0B820D0BFD0BED185D0BED0B4D0B8D0BBD0B020D0B1D0BED0BBD0B5D0B520D0BDD0B020D1
88D0BAD0B0D1842C20D187D0B5D0BC20D0BDD0B020D0BAD0B2D0B0D180D182D0B8D180D1832E
```

Искаженный открытый текст в формате Go (первые девять байт не образуют корректных символов UTF-8):

```
"\xd1J:j\xbc\xbf\xad\x92\xf1\x80ка его приходилась под самою кровлей высокого пятиэтажного дома и походила более на шкаф, чем на квартиру."
```

Теоретическое пояснение. В режиме CFB искаженный байт C1 участвует в двух преобразованиях. При дешифровании первого байта вычисляется P1 = C1 ⊕ MSB8(DES_K(C0)), поэтому инверсия одного бита C1 изменяет ровно один бит P1. Затем искаженный байт C1 попадает в 64-битный регистр обратной связи и участвует в формировании гаммы для следующих восьми байт P2–P9, вследствие чего эти байты искажаются псевдослучайным образом. После восьми сдвигов по 8 бит (в сумме 64 бита) байт C1 вытесняется из регистра, и начиная с P10 расшифрование снова становится верным (самосинхронизация режима). В блочных терминах первый 64-битный блок искажен на 29 бит, во втором блоке изменен только первый байт (1 бит), а блоки 3–25 не изменены. Схема распространения ошибки приведена на рисунке 4.

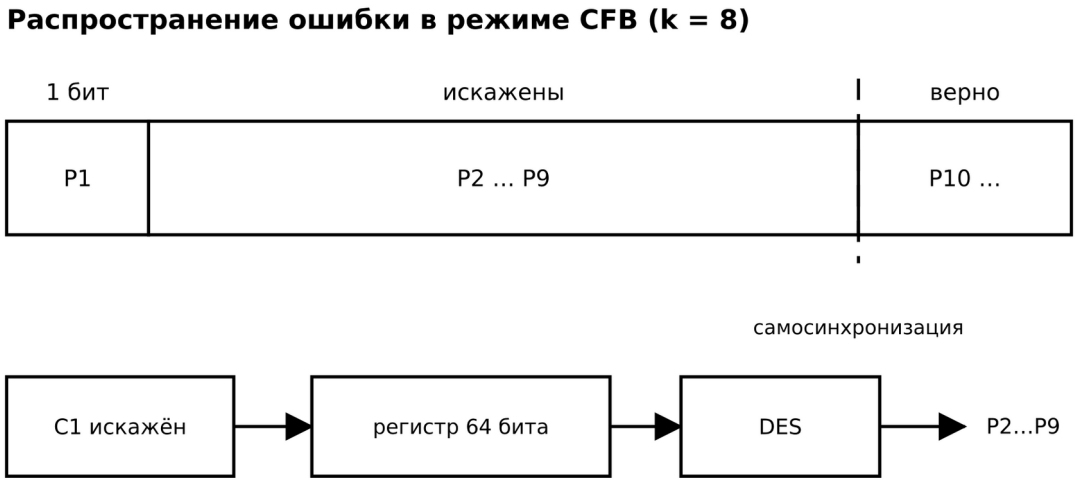


Рисунок 4 – Распространение ошибки при искажении байта C1

Таким образом, в отличие от режима ECB, где искажение одного бита шифрограммы затрагивает только один блок открытого текста, и режима CBC, где искажаются два блока, в CFB ошибка распределяется по девяти байтам (текущий байт — ровно один бит, следующие восемь байт — существенно), после чего режим самовосстанавливается. Это объясняется тем, что регистр обратной связи имеет длину 64 бита: искаженный байт влияет на гамму до тех пор, пока не будет полностью вытеснен из регистра.

5. Открытый текст с идентичными блоками и соответствующая шифрограмма.

Для проверки свойства обратной связи был взят открытый текст `BLOCK123BLOCK123`, содержащий два одинаковых 64-битных блока:

```
M1 = M2 = 424C4F434B313233
```

При шифровании в режиме CFB блоки шифрограммы различаются, поскольку регистр перед обработкой второго блока содержит результат обратной связи от первого блока:

```
CFB: C1 = 1968FE350CE2FB08
```

```
     C2 = EBB104E1BFB03DB4
     C1 == C2: false
```

Для сравнения тот же 16-байтовый текст был зашифрован в режимах ECB и CBC с тем же ключом и начальным вектором C0:

```
ECB: C1 = 5162DCB76F8C80C8
     C2 = 5162DCB76F8C80C8
     C1 == C2: true


CBC: C1 = 2EB753F919D6F394
     C2 = 025F6E41E95390C9
     C1 == C2: false
```

Полные шифрограммы (HEX):

```
CFB: 1968FE350CE2FB08EBB104E1BFB03DB4
ECB: 5162DCB76F8C80C85162DCB76F8C80C8
CBC: 2EB753F919D6F394025F6E41E95390C9
```

Теоретическое пояснение. В режиме ECB каждый блок шифруется независимо: C1 = DES(K, M1), C2 = DES(K, M2). Поскольку M1 = M2 и ключ одинаков, получаются равные блоки C1 = C2. В режиме CFB первый байт шифртекста определяется как C1 = P1 ⊕ MSB8(DES_K(C0)), а при обработке следующих байтов в регистр уже занесен предыдущий шифртекст. Поэтому даже при одинаковых блоках открытого текста входы функции DES для первого и второго блоков различаются, и блоки шифрограммы получаются разными. Режим CBC также скрывает равенство блоков, однако делает это за счет гаммирования с предыдущим блоком шифртекста, а CFB — за счет обратной связи по шифртексту на уровне k-битных сегментов. Дополнение PKCS#7 для данного теста не потребовалось: длина текста кратна 8 байтам, а в CFB дополнение не используется в принципе.

Сводное сравнение рассмотренных режимов приведено в таблице 1.

Таблица 1 – Сравнение режимов ECB, CBC и CFB

Характеристика	**ECB**	**CBC**	**CFB**
Единица обработки	64-битный блок	64-битный блок	k-битный сегмент (k = 8)
Дополнение PKCS#7	требуется	требуется	не требуется
Начальный вектор	не используется	C0 (64 бита)	C0 (64 бита)
Шифрование	Ci = E_K(Mi)	Ci = E_K(Mi ⊕ Ci-1)	Ci = Pi ⊕ MSB_k(E_K(Ri-1))
Дешифрование	Mi = D_K(Ci)	Mi = Ci-1 ⊕ D_K(Ci)	Pi = Ci ⊕ MSB_k(E_K(Ri-1))
Одинаковые блоки открытого текста	одинаковый шифртекст	разные блоки	разные блоки
Ошибка 1 бита	только свой блок	весь блок + 1 бит в следующем	1 бит в текущем байте + 8 следующих байт, затем самосинхронизация
Параллелизм	есть	шифрование — нет, дешифрование — да	нет
Длина шифрограммы	кратна 8 байтам (с PKCS#7)	кратна 8 байтам (с PKCS#7)	равна длине открытого текста

6. Выводы по работе.

1. Изучен режим CFB алгоритма DES, являющийся потоковым режимом с обратной связью по шифртексту. При k = 8 открытый текст обрабатывается побайтово; функции EncryptCFB и DecryptCFB реализованы на основе одной и той же функции EncryptBlock, а дополнение PKCS#7 не требуется.

2. Базовый тест подтвердил корректность реализации: открытый текст длиной 204 байта после шифрования в режиме CFB дал шифрограмму длиной 204 байта, которая после дешифрования совпала с исходным текстом. Начальный вектор C0 = 0x747786D1489B3A55 сформирован генератором ПСЧ из лабораторной работы №3; для реальных систем такой линейный конгруэнтный генератор непригоден, необходимо использовать криптографически стойкий датчик.

3. Экспериментально подтверждено, что CFB скрывает одинаковые блоки открытого текста: для текста `BLOCK123BLOCK123` получены различные блоки C1 = 1968FE350CE2FB08 и C2 = EBB104E1BFB03DB4,

тогда как в режиме ECB они совпадают (C1 = C2 = 5162DCB76F8C80C8). Это достигается за счет обратной связи по шифртексту.

4. Исследовано распространение ошибки в режиме CFB. Инвертирование одного бита первого байта шифрограммы привело к искажению ровно одного бита первого байта открытого текста, искажению следующих восьми байт (P2–P9) и полному восстановлению начиная с P10. В блочных терминах первый 64-битный блок искажен на 29 бит, второй — на 1 бит, остальные блоки не изменены. Таким образом, ошибка в CFB распределяется по девяти байтам, но затем режим самосинхронизируется; это отличается от ECB (один блок) и CBC (два блока).

5. Режим CFB не устраняет недостатки самого алгоритма DES: эффективная длина ключа остается равной 56 битам, сохраняются слабые ключи, а режим не обеспечивает имитостойкости и целостности данных. Кроме того, шифрование и дешифрование в CFB выполняются последовательно, без распараллеливания, а начальный вектор C0 должен быть уникальным для каждого сообщения и передаваться получателю.